

Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System — Technical Laboratory Report

PlatformIO | STM32Cube | STM32F103C8 Blue Pill | FreeRTOS | Wokwi

1. Problem and Requirements

For this laboratory, I built a simulated room monitor that runs on an STM32 Blue Pill. The device samples temperature, humidity, and relative light; detects motion; shows one selected measurement on an OLED; sounds an alarm whenever the temperature leaves its normal range; and drops into an inactive state after a stretch with no motion.

The required temperature limits are 18 °C and 30 °C, and both endpoints count as normal — the alarm only fires low below 18 °C and high above 30 °C. The implementation is built on the STM32Cube framework with native FreeRTOS APIs, not the Arduino framework.

Wokwi translates the physical system into a virtual Blue Pill circuit with a DHT22, a photoresistor module, a PIR sensor, a KY-040 encoder, an SSD1306 OLED, and a buzzer. The LDR output is expressed as a relative percentage rather than calibrated lux.

One boundary worth stating up front: the DHT22 is wired up in the Wokwi diagram, but the current sensor code doesn't actually communicate with it. It returns fixed placeholder readings of 25.4 °C and 61.2% RH, and this report is careful to distinguish those placeholders from real sensor acquisition.

2. System Architecture and Design

Hardware architecture

Device	Pin / interface	Role
STM32F103C8 Blue Pill	MCU; internal HSI clock configured at 8 MHz	Runs application tasks and FreeRTOS

Device	Pin / interface	Role
Photoresistor module	PA0, ADC1 channel 0	Relative ambient light input
DHT22	PA1 in the circuit	Intended temperature and humidity input; currently not read by firmware
PIR sensor	PA2	Motion input
KY-040 encoder	PA3 CLK, PA4 DT, PA5 switch	Rotary page selection; firmware samples CLK and DT, but does not read the switch
SSD1306 OLED	PB6 SCL, PB7 SDA, I2C address 0x3C	Displays the selected page
Buzzer	PA8, TIM1 channel 1 PWM	Temperature alarm output
Serial monitor	PA9 TX, PA10 RX, USART1 at 115200 baud	Startup and task diagnostics

The MCU clock setup uses the internal 8 MHz HSI without a PLL. The HAL millisecond tick runs on TIM4 because the FreeRTOS port owns SysTick. The OLED communicates over a project-local bit-banged I2C implementation on PB6/PB7, and the buzzer is set up for a nominal 1 kHz PWM signal.

Software architecture

The application is split across sensor, display, input, alarm, motion, system-state, and RTOS-object modules. The pure decision functions for temperature alarms, encoder navigation, and system-state transitions are kept apart from the peripheral access code so they can be compiled and tested natively.

```

SensorTask --+--> display sensor queue --> DisplayTask --> SSD1306
              +--> alarm sensor queue --> AlarmTask --> buzzer
InputTask  -----> display-mode queue --> DisplayTask
MotionTask -----> EVENT_MOTION -----> StateTask
StateTask  -----> EVENT_ACTIVE -----> other tasks
All task diagnostics -----> USART1, guarded by serialMutex

```

DisplayTask is the only task that touches the OLED at runtime. SensorTask publishes each sensor record to separate display and alarm queues because a queue receive consumes the item — publishing once

would leave the two consumers fighting over it. The event group carries the ACTIVE, MOTION, and ALARM status flags, and the serial mutex protects the shared USART1 transmit path.

State machine

```
Boot -> ACTIVE
ACTIVE -- 15 seconds without motion --> INACTIVE
INACTIVE -- motion detected -----> ACTIVE
```

The system boots into ACTIVE. StateTask owns EVENT_ACTIVE and applies the 15-second inactivity rule. While the system is inactive, MotionTask keeps sampling the PIR so any activity can wake it up; SensorTask skips acquisition; InputTask keeps polling but throws away mode changes; DisplayTask blanks the screen the next time it checks the state; and AlarmTask silences the buzzer.

3. FreeRTOS Architecture

All six application tasks are created in main.cpp with explicit priorities. The periodic input, motion, and sensor work uses vTaskDelayUntil() so each run stays aligned to a fixed period instead of letting the delay accumulate after every task body. Tasks waiting on queues or event groups sit in the Blocked state until their data, event, or timeout arrives.

Task	Responsibility	Trigger / period	Priority	IPC	Typical blocked condition
SensorTask	Read LDR and current temperature/humidity values; publish SensorData to two consumer queues	2 s, vTaskDelayUntil()	2	Two sensor queues; reads EVENT_ACTIVE and EVENT_MOTION	Periodic delay; queue send can wait up to 50 ms
DisplayTask	Render the selected measurement; sole OLED owner	Sensor update or queue timeout	1	Sensor queue and mode queue	Waits up to 2.5 s for a sensor queue item

Task	Responsibility	Trigger / period	Priority	IPC	Typical blocked condition
InputTask	Decode encoder CLK/DT and select a display page while ACTIVE	5 ms, vTaskDelayUntil()	3	Display-mode queue; reads EVENT_ACTIVE	5 ms periodic delay
MotionTask	Sample the PIR and mirror its level to EVENT_MOTION	100 ms, vTaskDelayUntil()	3	System event group	100 ms periodic delay
AlarmTask	Evaluate temperature, drive buzzer PWM, publish EVENT_ALARM	Sensor data; 500 ms queue timeout	2	Alarm sensor queue and system event group	Waits up to 500 ms for sensor data
StateTask	Apply ACTIVE/INACTIVE transitions and inactivity timeout	Motion event or 15 s timeout	2	Waits on EVENT_MOTION; owns EVENT_ACTIVE	Event wait; 500 ms delay after detecting asserted motion

The priorities follow the latency requirements. Encoder and PIR input run at priority 3 because those signals can change on a short timescale. Sensor, alarm, and state processing sit at priority 2. OLED updates run at priority 1, since a small delay on the display is acceptable and a slow display operation shouldn't outrank input or sampling work.

In FreeRTOS terms, a task that is eligible to run but waiting for the CPU is Ready; a task waiting on a delay, queue item, or event is Blocked. The periodic tasks finish their finite work and return to Blocked. All tasks and RTOS objects are created dynamically from the configured FreeRTOS heap.

4. Implementation

SensorData holds temperature, humidity, lightLevel, and motionDetected. The current DHT22 function just assigns constants, while the LDR is sampled through ADC1 and scaled to an integer from 0 to 100. That scale is relative brightness and must not be presented as lux.

DisplayMode is an enumeration with TEMPERATURE, HUMIDITY, LIGHT, and MOTION values. InputTask watches for a falling edge on encoder CLK and compares DT to decide between forward and reverse navigation, then posts the new mode to a queue; DisplayTask performs the actual OLED update. The encoder switch input is configured on PA5 but isn't processed yet.

Temperature evaluation is deliberately hardware-independent: values below 18 °C map to LOW_TEMPERATURE, values above 30 °C map to HIGH_TEMPERATURE, and everything from 18 °C through 30 °C maps to NORMAL. AlarmTask drives TIM1 channel 1 PWM on PA8 whenever the system is ACTIVE and the temperature is out of range.

The UART mutex is taken inside the _write() retarget, around each HAL_UART_Transmit() call, once the scheduler has started; startup output is emitted before the scheduler runs. The mutex guards the peripheral transmit path — it doesn't wrap an entire printf call as one indivisible record.

5. Verification and Testing

Build and unit tests

Verification	Recorded result	Scope / caveat
pio run	PASS; last recorded build used 50.2% RAM and 50.4% flash	Firmware compiles and links for bluepill_f103c8
pio test -e native	13 tests passed	Five alarm, four navigation, and four state-logic cases; not peripheral or task integration tests
pio check -e bluepill_f103c8	PASS; 0 high, 0 medium, 10 low findings	Remaining low findings are documented in docs/static-analysis.md
pio check -e native	PASS; no defects reported	Checks only hardware-independent *_logic.cpp files

Wokwi observations

The user-supplied Wokwi screenshot shows the OLED on the HUMIDITY page and a live serial terminal with sample lines reading 25.40 °C, 61.25% humidity, 15% light, and no motion. That's useful evidence a simulation was running, but it doesn't establish all of FT-01 through FT-10.

There's also a discrepancy worth flagging: the current source assigns humidity 61.2f and formats serial output to two decimal places, which should display 61.20%, yet the screenshot shows 61.25%. The screenshot may come from another or earlier firmware state, so it's recorded here as an observed screenshot rather than a verified output of the current checked source.

Test	Required stimulus	Current record
FT-01 / FT-02	Change DHT22 temperature and humidity; confirm the OLED and terminal update	Not verified against current source; DHT22 values are constants
FT-03	Move the LDR from dark to bright and confirm percentage direction	Screenshot shows 15%; dark/bright comparison not recorded
FT-04 to FT-06	Trigger PIR and navigate OLED pages in both directions	Not recorded as a complete test sequence
FT-07	Exercise both alarm limits and normal return	Logic boundaries covered in unit tests; interactive buzzer verification pending
FT-08 to FT-10	Observe inactivity, 15 s timeout, and motion wake-up	Interactive observations pending

No Wokwi run has been independently reproduced for this report. The repository checklist in docs/functional-verification.md keeps these runtime tests pending until each stimulus is actually performed and its real result recorded.

Requirements traceability

The matrix below connects each functional requirement to the code that implements it and to the available verification evidence. A pending Wokwi result is not treated as a pass, and the DHT22 requirements are explicitly marked incomplete because the current firmware still uses placeholder values.

Requirement	Implementation	Verification evidence	Status
FR-01 Temperature measurement	SensorTask and the DHT22 acquisition path in sensors.cpp	FT-01 pending; current DHT22 function returns a fixed value	Incomplete
FR-02 Humidity measurement	SensorTask and the DHT22 acquisition path in sensors.cpp	FT-02 pending; current DHT22 function returns a fixed value	Incomplete
FR-03 Ambient-light measurement	SensorTask reads ADC1 channel 0 and scales the LDR to 0-100%	FT-03 pending; ADC conversion and scaling are implemented	Implemented; pending Wokwi evidence
FR-04 Motion detection	MotionTask samples the PIR and updates EVENT_MOTION	FT-04 pending; motion logic is implemented	Implemented; pending Wokwi evidence
FR-05 OLED display	DisplayTask exclusively owns the SSD1306 and renders one selected mode	FT-05 pending; display mode behavior is implemented	Implemented; pending Wokwi evidence
FR-06 Rotary encoder navigation	InputTask decodes CLK/DT and uses nextDisplayMode()/previousDisplayMode()	Four native navigation tests pass; FT-05 and FT-06 are pending	Implemented; partially verified
FR-07 Temperature alarm	AlarmTask evaluates temperature and controls TIM1 PWM on the buzzer	Five native alarm-boundary tests pass; FT-07 is pending	Implemented; partially verified
FR-08 Activity state	StateTask manages EVENT_ACTIVE with ACTIVE and INACTIVE states	Four native state tests pass; runtime	Implemented; partially verified

Requirement	Implementation	Verification evidence	Status
		state observations are pending	
FR-09 Automatic inactivity	StateTask clears EVENT_ACTIVE after the 15-second timeout	Native timeout test passes; FT-08 and FT-09 are pending	Implemented; partially verified
FR-10 Automatic reactivation	MotionTask keeps EVENT_MOTION active while inactive; StateTask restores ACTIVE	Native reactivation test passes; FT-10 is pending	Implemented; pending Wokwi evidence

6. Static Code Analysis

I ran PlatformIO's cppcheck for both the firmware and native environments. The firmware environment came back with no high- or medium-severity findings and ten low-severity ones, while the native logic-only check reported no defects. The firmware findings are stylistic, tied to STM32 CMSIS/HAL register macros and the required HAL callback signature — the details and accepted resolutions are recorded in docs/static-analysis.md.

The native check intentionally skips STM32-only sources such as the custom port, so its clean result applies to the hardware-independent decision logic rather than the complete firmware source set. PlatformIO also emitted some nonfatal toolchain-define extraction warnings during the native check.

7. Engineering Discussion

The main architectural trade-off here is splitting sensor fan-out across two queues. It avoids a race in which DisplayTask and AlarmTask would otherwise compete to consume the same queue item, at the cost of publishing the same record twice. I used the event group for shared state flags because several tasks need to read the current ACTIVE and MOTION levels, and a mutex on the UART because several task contexts emit diagnostics.

One consequence of the design: the display task waits up to 2.5 seconds on its sensor queue before checking system activity and redrawing, so a mode change or an inactive transition may not show up immediately. InputTask keeps polling while inactive but discards mode changes, and the PA5 encoder switch isn't consumed. These are known limitations, not verified feature claims.

The project uses a custom FreeRTOS Cortex-M3 port, chosen for how it behaves in the Wokwi simulator — a simulator-specific engineering decision, since physical hardware testing hasn't been documented. The most significant current functional limitation is the unimplemented DHT22 protocol, which blocks real temperature/humidity updates and any meaningful simulated temperature alarm changes.

8. Conclusion

All told, the project delivers a modular, concurrent STM32 FreeRTOS application: six distinct tasks, queues, an event group, a serial mutex, an OLED display path, PIR-based state logic, and independently testable alarm, navigation, and state decisions. The recorded firmware build and 13 native logic tests all pass.

That said, it isn't yet a complete functional demonstration of every lab requirement. The DHT22 driver is still a placeholder, and the full FT-01 through FT-10 Wokwi verification record remains incomplete. The next engineering steps are to implement real DHT22 acquisition, sort out the screenshot/source humidity discrepancy, and record each interactive Wokwi test against the current firmware.